

Додаток 2. Результати тестування

Valid Basic

examples/valid-basic.signal

```
1 PROGRAM HELLO;  
2 CONST  
3   X = '42';  
4 BEGIN  
5 END.
```

Output

Tokens

Code	Row	Col	Type	Value
300	1	1	Keyword	PROGRAM
1000	1	9	Identifier	HELLO
59	1	14	Delimiter	;
301	2	1	Keyword	CONST
1001	3	3	Identifier	X
61	3	5	Delimiter	=
39	3	7	Delimiter	'
500	3	8	Literal	42
39	3	10	Delimiter	'
59	3	11	Delimiter	;
302	4	1	Keyword	BEGIN
303	5	1	Keyword	END
46	5	4	Delimiter	.

Identifiers

Code	Value
1000	HELLO
1001	X

Literals

Code	Value
500	42

Syntax Tree

```
<signal-program>  
└(1)─<program>  
    └(2)─PROGRAM <procedure-identifier>; <block>.  
        └(14)─<identifier>  
            └(T)─HELLO  
        └(3)─<declarations> BEGIN <statements-list> END  
            └(5)─<constant-declarations>  
                └(6)─CONST <constant-declarations-list>  
                    └(7)─<constant-declaration> <constant-declarations-list>  
                        └(8)─<constant-identifier> = <constant>;  
                            └(13)─<identifier>  
                                └(T)─X  
                            └(9)─'<complex-number>'  
                                └(10)─<left-part> <right-part>  
                                    └(11)─<unsigned-integer>  
                                        └(T)─42  
                                    └(12)─<empty>  
                                └(7)─<empty>  
                        └(4)─<empty>
```

Declarations

Identifier	Kind	Type	Value
HELLO	Program		
X	Constant	Complex Integer	42 + i*0

X: section .data:
dd 42, 0

```
    section .text:
    global _start
_start:
    nop
    mov     eax, 60
    xor     edi, edi
    syscall
```

Valid Basic With Tabs

examples/valid-basic-with-tabs.signal

```
1 PROGRAM TABS;  
2 CONST  
3     X      =      '1';  
4     Y =      '2$EXP(3)';  
5     Z      =  '4,5';  
6 BEGIN  
7 END.
```

Output

Tokens

Code	Row	Col	Type	Value
300	1	1	Keyword	PROGRAM
1000	1	9	Identifier	TABS
59	1	13	Delimiter	;
301	2	1	Keyword	CONST
1001	3	5	Identifier	X
61	3	9	Delimiter	=
39	3	13	Delimiter	'
500	3	14	Literal	1
39	3	15	Delimiter	'
59	3	16	Delimiter	;
1002	4	5	Identifier	Y
61	4	7	Delimiter	=
39	4	9	Delimiter	'
501	4	10	Literal	2
256	4	11	Multi-Delimiter	\$EXP
40	4	15	Delimiter	(
502	4	16	Literal	3
41	4	17	Delimiter	)
39	4	18	Delimiter	'
59	4	19	Delimiter	;
1003	5	9	Identifier	Z
61	5	13	Delimiter	=
39	5	15	Delimiter	'
503	5	16	Literal	4
44	5	17	Delimiter	,
504	5	18	Literal	5
39	5	19	Delimiter	'
59	5	20	Delimiter	;
302	6	1	Keyword	BEGIN
303	7	1	Keyword	END
46	7	4	Delimiter	.

Identifiers

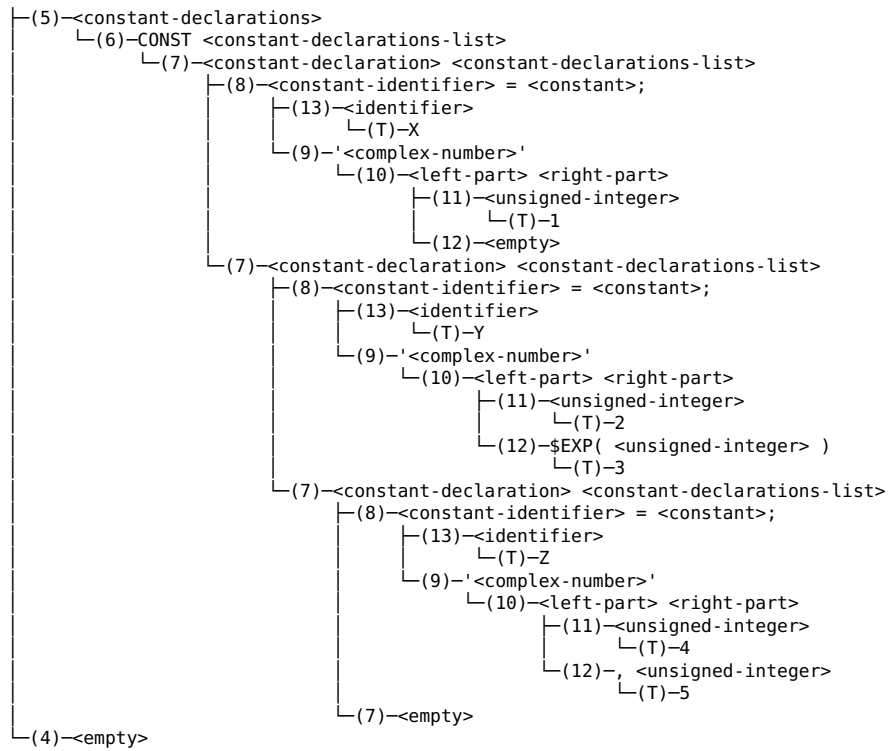
Code	Value
1000	TABS
1001	X
1002	Y
1003	Z

Literals

Code	Value
500	1
501	2
502	3
503	4
504	5

Syntax Tree

<signal-program>
└─(1)─<program>
 └─(2)─PROGRAM <procedure-identifier>; <block>.
 └─(14)─<identifier>
 └─(T)─TABS
 └─(3)─<declarations> BEGIN <statements-list> END



Declarations

Identifier	Kind	Type	Value
TABS	Program		
X	Constant	Complex Integer	1 + i*0
Y	Constant	Complex Float	-1.980 + i*0.282
Z	Constant	Complex Integer	4 + i*5

```

section .data:
X:    dd    1, 0
Y:    dd    -1.97998, 0.28224
Z:    dd    4, 5

```

```

section .text:
global _start
_start:
    nop
    mov    eax, 60
    xor    edi, edi
    syscall

```

Valid Comments

examples/valid-comments.signal

```
1 (* A lot of comments *)
2 PROGRAM COMMENTS;
3 CONST
4   (* Single-line comment *)
5   (*
6     Multi-line comment
7   *)
8   (**) (* *)
9   (* Multiple *) (* comments *) (* on *) (* the *) (* same *) (* line *)
10  (*****) ((*())*) (*;.* *) (* Comments with nested * and () *)
11  (* Invalid symbols are allowed! %#^@& *)
12 BEGIN (* on the same like as identifier *)
13 END (* between tokens *) .
```

Output

Tokens

Code	Row	Col	Type	Value
300	2	1	Keyword	PROGRAM
1000	2	9	Identifier	COMMENTS
59	2	17	Delimiter	;
301	3	1	Keyword	CONST
302	12	1	Keyword	BEGIN
303	13	1	Keyword	END
46	13	26	Delimiter	.

Identifiers

Code	Value
1000	COMMENTS

Literals

Code	Value
------	-------

Syntax Tree

```
<signal-program>
└(1)─<program>
    └(2)─PROGRAM <procedure-identifier>; <block>.
        └(14)─<identifier>
            └(T)─COMMENTS
        └(3)─<declarations> BEGIN <statements-list> END
            └(5)─<constant-declarations>
                └(6)─CONST <constant-declarations-list>
                    └(7)─<empty>
            └(4)─<empty>
```

Declarations

Identifier	Kind	Type	Value
COMMENTS	Program		

section .text:
global _start
_start:
nop
mov eax, 60
xor edi, edi
syscall

Valid Constants

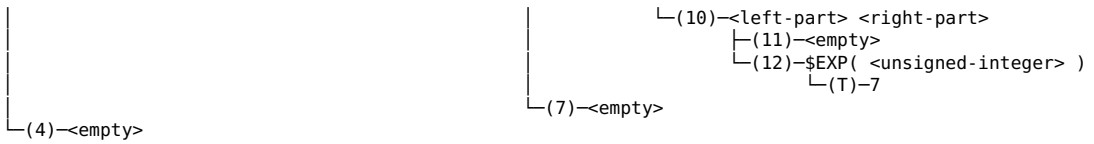
examples/valid-constants.signal

```
1 (* All ways to declare a constant within a grammar *)
2 PROGRAM CONSTANTS;
3 CONST
4   (* Single-line comment *)
5   A = '';
6   B = '1';
7   C = '2,3';
8   D = ',4';
9   E = '5$EXP(6)';
10  F = '$EXP(7)';
11 BEGIN
12   (*
13    Multi-line comment
14   *)
15 END.
```

Output

Tokens

Code	Row	Col	Type	Value
300	2	1	Keyword	PROGRAM
1000	2	9	Identifier	CONSTANTS
59	2	18	Delimiter	;
301	3	1	Keyword	CONST
1001	5	3	Identifier	A
61	5	5	Delimiter	=
39	5	7	Delimiter	'
39	5	8	Delimiter	'
59	5	9	Delimiter	;
1002	6	3	Identifier	B
61	6	5	Delimiter	=
39	6	7	Delimiter	'
500	6	8	Literal	1
39	6	9	Delimiter	'
59	6	10	Delimiter	;
1003	7	3	Identifier	C
61	7	5	Delimiter	=
39	7	7	Delimiter	'
501	7	8	Literal	2
44	7	9	Delimiter	,
502	7	10	Literal	3
39	7	11	Delimiter	'
59	7	12	Delimiter	;
1004	8	3	Identifier	D
61	8	5	Delimiter	=
39	8	7	Delimiter	'
44	8	8	Delimiter	,
503	8	9	Literal	4
39	8	10	Delimiter	'
59	8	11	Delimiter	;
1005	9	3	Identifier	E
61	9	5	Delimiter	=
39	9	7	Delimiter	'
504	9	8	Literal	5
256	9	9	Multi-Delimiter	\$EXP
40	9	13	Delimiter	(
505	9	14	Literal	6
41	9	15	Delimiter	)
39	9	16	Delimiter	'
59	9	17	Delimiter	;
1006	10	3	Identifier	F
61	10	5	Delimiter	=
39	10	7	Delimiter	'
256	10	8	Multi-Delimiter	\$EXP
40	10	12	Delimiter	(
506	10	13	Literal	7
41	10	14	Delimiter	)
39	10	15	Delimiter	'
59	10	16	Delimiter	;
302	11	1	Keyword	BEGIN
303	15	1	Keyword	END



Valid Edge Cases

examples/valid-edge-cases.signal

```
1 (* Edge cases within the grammar *)
2 PROGRAM EDGES;
3 CONST
4   AB12CD = '999,000';
5   Z = '100$EXP(200)';
6   LONGNAME123 = '0';
7   A = '0,0';
8 BEGIN
9 END.
```

Output

Tokens				
Code	Row	Col	Type	Value
300	2	1	Keyword	PROGRAM
1000	2	9	Identifier	EDGES
59	2	14	Delimiter	;
301	3	1	Keyword	CONST
1001	4	3	Identifier	AB12CD
61	4	10	Delimiter	=
39	4	12	Delimiter	'
500	4	13	Literal	999
44	4	16	Delimiter	,
501	4	17	Literal	000
39	4	20	Delimiter	'
59	4	21	Delimiter	;
1002	5	3	Identifier	Z
61	5	5	Delimiter	=
39	5	7	Delimiter	'
502	5	8	Literal	100
256	5	11	Multi-Delimiter	\$EXP
40	5	15	Delimiter	(
503	5	16	Literal	200
41	5	19	Delimiter	)
39	5	20	Delimiter	'
59	5	21	Delimiter	;
1003	6	3	Identifier	LONGNAME123
61	6	15	Delimiter	=
39	6	17	Delimiter	'
504	6	18	Literal	0
39	6	19	Delimiter	'
59	6	20	Delimiter	;
1004	7	3	Identifier	A
61	7	5	Delimiter	=
39	7	7	Delimiter	'
504	7	8	Literal	0
44	7	9	Delimiter	,
504	7	10	Literal	0
39	7	11	Delimiter	'
59	7	12	Delimiter	;
302	8	1	Keyword	BEGIN
303	9	1	Keyword	END
46	9	4	Delimiter	.

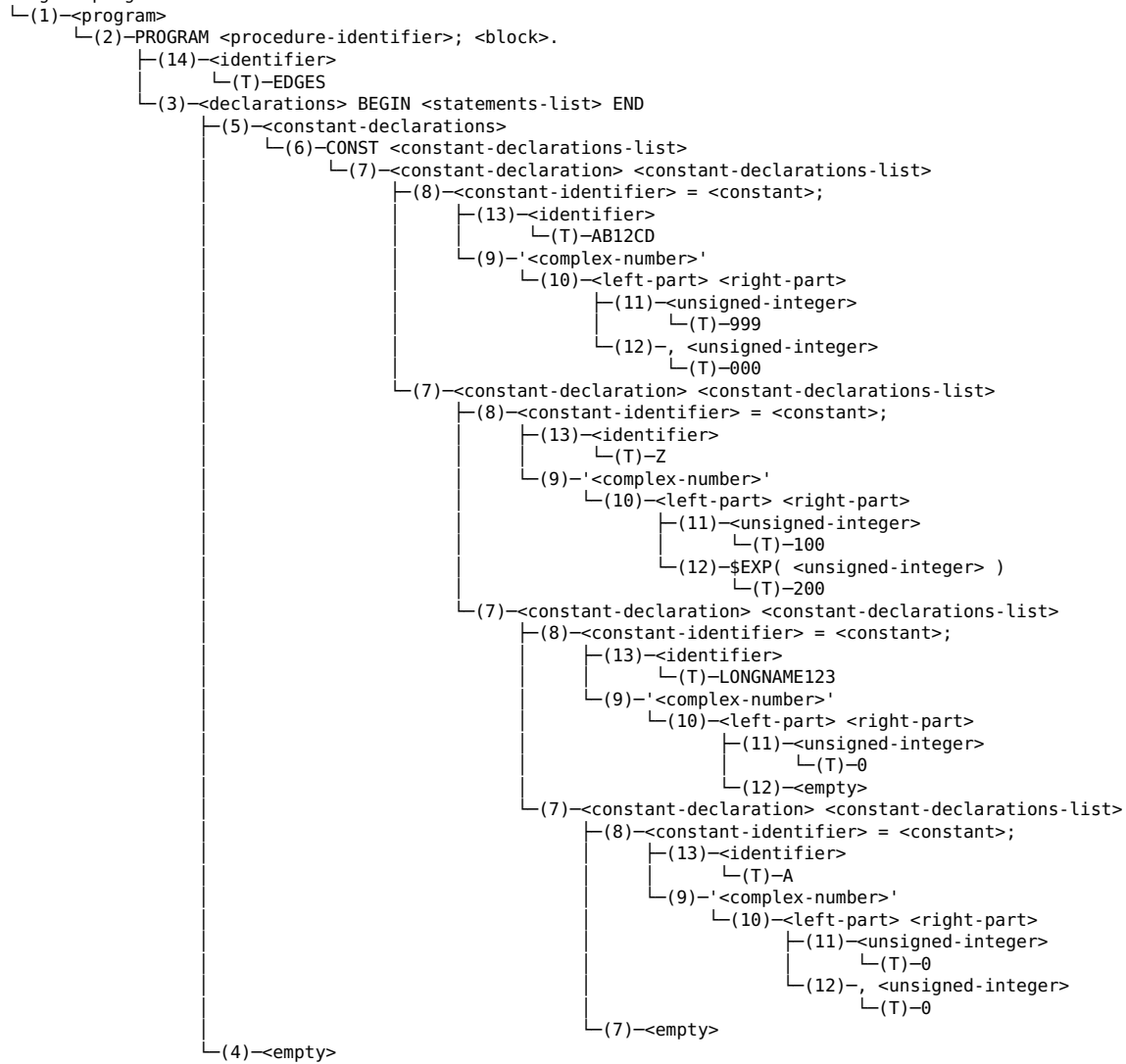
Identifiers	
Code	Value
1000	EDGES
1001	AB12CD
1002	Z
1003	LONGNAME123
1004	A

Literals	
Code	Value
500	999
501	000
502	100

503	200
504	0

Syntax Tree

```
<signal-program>
```



Declarations

Identifier	Kind	Type	Value
A	Constant	Complex Integer	0 + i*0
AB12CD	Constant	Complex Integer	999 + i*0
EDGES	Program		
LONGNAME123	Constant	Complex Integer	0 + i*0
Z	Constant	Complex Float	48.719 + i*-87.330

```

section .data:
AB12CD: dd    999, 0
Z:      dd    48.7188, -87.3297
LONGNAME123: dd    0, 0
A:      dd    0, 0

```

```

section .text:
global _start
_start:
nop
mov     eax, 60
xor     edi, edi
syscall

```

Valid Empty

examples/valid-empty.signal

```
1 (* Empty program *)
2 PROGRAM EMPTY;
3 BEGIN
4 END.
```

Output

Tokens

Code	Row	Col	Type	Value
300	2	1	Keyword	PROGRAM
1000	2	9	Identifier	EMPTY
59	2	14	Delimiter	;
302	3	1	Keyword	BEGIN
303	4	1	Keyword	END
46	4	4	Delimiter	.

Identifiers

Code	Value
1000	EMPTY

Literals

Code	Value

Syntax Tree

<signal-program>
└(1)─<program>
 └(2)─PROGRAM <procedure-identifier>; <block>.
 └(14)─<identifier>
 └(T)─EMPTY
 └(3)─<declarations> BEGIN <statements-list> END
 └(5)─<constant-declarations>
 └(6)─<empty>
 └(4)─<empty>

Declarations

Identifier	Kind	Type	Value
EMPTY	Program		

section .text:
global _start
_start:
 nop
 mov eax, 60
 xor edi, edi
 syscall

Semantic Error Const Redefine

examples/semantic-error-const-redefine.signal

```
1 PROGRAM HELLO;  
2 CONST  
3   X = '1';  
4   X = '2';  
5 BEGIN  
6 END.
```

Output

Semantics | Error [4:3]: Identifier 'X' is already declared at [3:3]

Tokens

Code	Row	Col	Type	Value
300	1	1	Keyword	PROGRAM
1000	1	9	Identifier	HELLO
59	1	14	Delimiter	;
301	2	1	Keyword	CONST
1001	3	3	Identifier	X
61	3	5	Delimiter	=
39	3	7	Delimiter	'
500	3	8	Literal	1
39	3	9	Delimiter	'
59	3	10	Delimiter	;
1001	4	3	Identifier	X
61	4	5	Delimiter	=
39	4	7	Delimiter	'
501	4	8	Literal	2
39	4	9	Delimiter	'
59	4	10	Delimiter	;
302	5	1	Keyword	BEGIN
303	6	1	Keyword	END
46	6	4	Delimiter	.

Identifiers

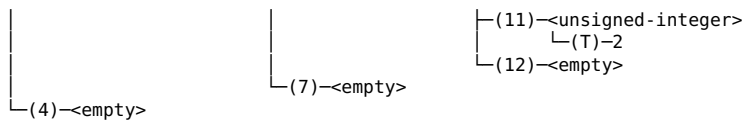
Code	Value
1000	HELLO
1001	X

Literals

Code	Value
500	1
501	2

Syntax Tree

```
<signal-program>  
└(1)─<program>  
    └(2)─PROGRAM <procedure-identifier>; <block>.  
        └(14)─<identifier>  
            └(T)─HELLO  
        └(3)─<declarations> BEGIN <statements-list> END  
            └(5)─<constant-declarations>  
                └(6)─CONST <constant-declarations-list>  
                    └(7)─<constant-declaration> <constant-declarations-list>  
                        └(8)─<constant-identifier> = <constant>;  
                            └(13)─<identifier>  
                                └(T)─X  
                            └(9)─'<complex-number>'  
                                └(10)─<left-part> <right-part>  
                                    └(11)─<unsigned-integer>  
                                        └(T)─1  
                                    └(12)─<empty>  
                                └(7)─<constant-declaration> <constant-declarations-list>  
                                    └(8)─<constant-identifier> = <constant>;  
                                        └(13)─<identifier>  
                                            └(T)─X  
                                        └(9)─'<complex-number>'  
                                            └(10)─<left-part> <right-part>
```



Declarations

Identifier	Kind	Type	Value
HELLO X	Program Constant	Complex Integer	1 + i*0

Semantic Error Mixed

examples/semantic-error-mixed.signal

```
1 PROGRAM HELLO;
2 CONST
3   X = '1';
4   X = '2';
5   HELLO = '3,4';
6 BEGIN
7 END.
```

Output

Semantics | Error [4:3]: Identifier 'X' is already declared at [3:3]
Semantics | Error [5:3]: Identifier 'HELLO' is already declared at [1:9]

Tokens

Code	Row	Col	Type	Value
300	1	1	Keyword	PROGRAM
1000	1	9	Identifier	HELLO
59	1	14	Delimiter	;
301	2	1	Keyword	CONST
1001	3	3	Identifier	X
61	3	5	Delimiter	=
39	3	7	Delimiter	'
500	3	8	Literal	1
39	3	9	Delimiter	'
59	3	10	Delimiter	;
1001	4	3	Identifier	X
61	4	5	Delimiter	=
39	4	7	Delimiter	'
501	4	8	Literal	2
39	4	9	Delimiter	'
59	4	10	Delimiter	;
1000	5	3	Identifier	HELLO
61	5	9	Delimiter	=
39	5	11	Delimiter	'
502	5	12	Literal	3
44	5	13	Delimiter	,
503	5	14	Literal	4
39	5	15	Delimiter	'
59	5	16	Delimiter	;
302	6	1	Keyword	BEGIN
303	7	1	Keyword	END
46	7	4	Delimiter	.

Identifiers

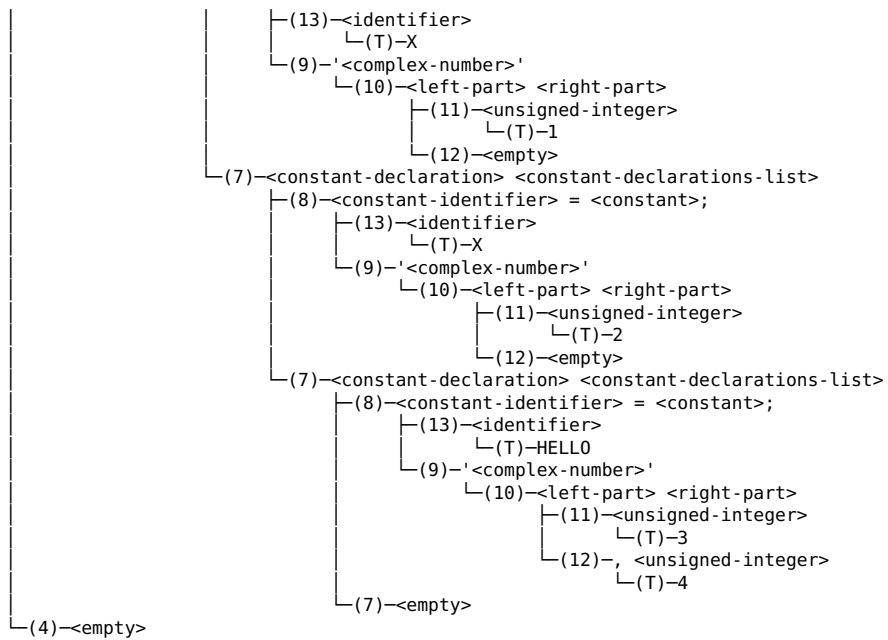
Code	Value
1000	HELLO
1001	X

Literals

Code	Value
500	1
501	2
502	3
503	4

Syntax Tree

```
<signal-program>
└(1)─<program>
    └(2)─PROGRAM <procedure-identifier>; <block>.
        └(14)─<identifier>
            └(T)─HELLO
        └(3)─<declarations> BEGIN <statements-list> END
            └(5)─<constant-declarations>
                └(6)─CONST <constant-declarations-list>
                    └(7)─<constant-declaration> <constant-declarations-list>
                        └(8)─<constant-identifier> = <constant>;
```



Declarations

Identifier	Kind	Type	Value
HELLO X	Program Constant	Complex Integer	1 + i*0

Semantic Error Program Redefine

examples/semantic-error-program-redefine.signal

```
1 PROGRAM X;
2 CONST
3   X = '42';
4 BEGIN
5 END.
```

Output

Semantics | Error [3:3]: Identifier 'X' is already declared at [1:9]

Tokens

Code	Row	Col	Type	Value
300	1	1	Keyword	PROGRAM
1000	1	9	Identifier	X
59	1	10	Delimiter	;
301	2	1	Keyword	CONST
1000	3	3	Identifier	X
61	3	5	Delimiter	=
39	3	7	Delimiter	'
500	3	8	Literal	42
39	3	10	Delimiter	'
59	3	11	Delimiter	;
302	4	1	Keyword	BEGIN
303	5	1	Keyword	END
46	5	4	Delimiter	.

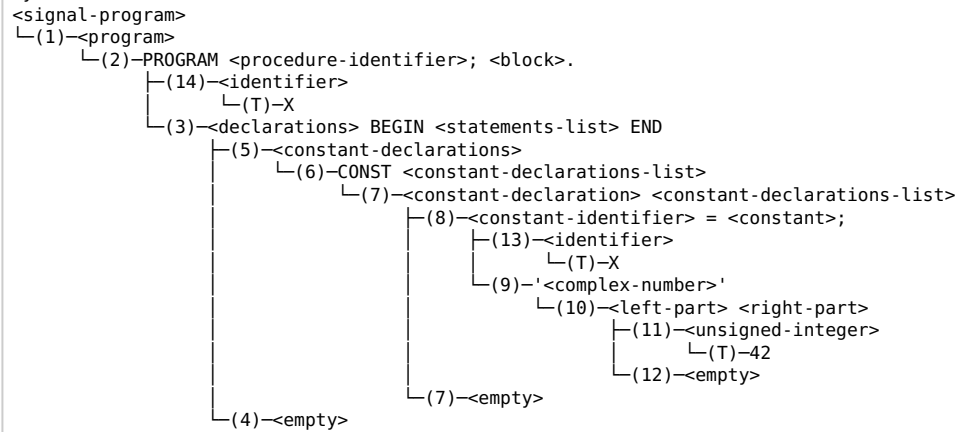
Identifiers

Code	Value
1000	X

Literals

Code	Value
500	42

Syntax Tree



Declarations

Identifier	Kind	Type	Value
X	Program		